

Projet - Inférence automatique de la relation entre deux phrases

AKIBODE Yayrale

Professeur: Adrien Guille

M2 MALIA, Université Lumière Lyon 2

4 janvier 2026

Résumé

Ce projet explore et compare les performances de modèles de traitement du langage naturel, à savoir CamemBERT 2.0, CamemBERTa 2.0 et LLaMA 3, dans le cadre d'une tâche d'inférence de relations entre deux phrases. L'objectif est de prédire, pour chaque paire de phrases, l'une des trois classes suivantes : contradiction, entailment ou neutral. Après une exploration des données montrant un jeu de données équilibré et exempt d'anomalies, nous avons étudié les différences architecturales entre CamemBERT (basé sur RoBERTa) et CamemBERTa (basé sur DeBERTa v3). Les modèles ont ensuite été fine-tunés avec LoRA, une méthode utilisant des matrices de rang faible pour réduire le nombre de paramètres à entraîner, ce qui a permis d'obtenir de très bonnes performances avec CamemBERTa 2.0 et LLaMA 3 1B. Par ailleurs, nous avons expérimenté différents modes d'apprentissage en contexte qui sont le 0-shot, le few-shot et le Chain-of-thought sur LLaMA 3 3B-Instruct. Les résultats montrent que ces modes sont généralement moins performants que le fine-tuning, en particulier pour la prédiction de la classe "neutral", mais que le mode Chain-of-thought améliore sensiblement la capacité du modèle à traiter cette classe ambiguë. Ainsi, cette étude met en lumière l'efficacité du fine-tuning avec LoRA et l'intérêt d'un raisonnement explicite pour améliorer la classification sur des classes difficiles dans le cadre de l'inférence de relations entre phrases.

Introduction.

Le traitement automatique du langage naturel connaît un essor considérable grâce aux modèles de type Transformer, capables de comprendre et de générer du texte avec une grande précision. Dans ce contexte, les modèles encodeurs tels que CamemBERT 2.0 et CamemBERTa 2.0, ainsi que les modèles décodeurs comme LLaMA 3, offrent des architectures complémentaires pour différentes tâches de compréhension et de génération de texte.

L'objectif de ce projet est d'explorer et de comparer ces modèles dans le cadre d'une tâche d'inférence de relations entre deux phrases. De fait, nous voulons prédire pour chaque paire de phrases, l'une des trois classes suivantes : contradiction, entailment ou neutral. Nous commencerons par une exploration des données afin de détecter d'éventuelles anomalies, puis nous effectuerons une veille documentaire pour mieux comprendre les particularités de chaque modèle.

Pour adapter ces modèles à notre tâche, nous utiliserons la méthode LoRA, qui permet de réduire le nombre de paramètres à fine-tuner grâce à des matrices de rang faible. Nous expérimenterons également différents modes d'apprentissage en contexte qui sont le 0-shot, le few-shot et le Chain-of-thought sur le modèle LLaMA 3 3B-Instruct, afin d'évaluer leur capacité à généraliser.

À la fin de ce projet, nous souhaitons mettre en évidence les différences entre CamemBERT et CamemBERTa, l'intérêt de la méthode LoRA, ainsi que l'apport du mode Chain-of-thought pour améliorer la performance sur des tâches complexes.

1 Exploration du dataset d'apprentissage.

Notre dataset d'apprentissage est constitué de deux tableaux (*train* : 5010 observations et *test* : 2490 observations). Chaque table comprend trois colonnes.

- La prémisse : c'est une vérité établie. Elle constitue le contexte sur lequel nous appuyons pour tester l'hypothèse.
- L'hypothèse : c'est la phrase à tester afin de déterminer sa relation avec la prémisse.
- Le label : il s'agit de la classe qui indique la relation entre les deux phrases.

Équilibre des classes.

L'équilibre des classes est très important dans la classification. Il permet d'avoir une bonne accuracy car aucune classe n'est dominante. Sur le graphe ci-dessous, les classes sont bien équilibrées. Cela veut dire que les modèles n'auront pas tendance à prédire une classe plus que les autres.

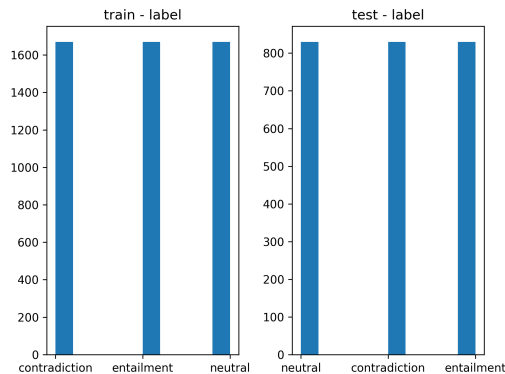


FIGURE 1 – Équilibre des classes.

Longueur des phrases.

L'étude des longueurs de phrases montrent que l'hypothèse est deux fois plus courte que la prémisse et ceci pour toutes les classes. Cette information est très importante car le modèle ne pourra pas exploiter la longueur des phrases pour tricher. En effet, dans la littérature une conséquence est plus courte. Une phrase neutre est longue car elle ajoute des informations extrinsèques, tandis qu'une contradiction n'a pas de longueur explicite.

Mots les plus fréquents.

Dans cette sous-section, nous analysons les dix (10) premiers mots les plus fréquents ormis les mots vides de chaque classe. Les remarques relevées concernent plus la classe de contradiction. Les mots les utilisés sont des mots négatifs. Pour les autres classes les mots les plus utilisés sont plutôt les mêmes.

```
--- Most common Words - Entailment ---
[('a', 291), ('plus', 74), ('beaucoup', 51), ('j'ai', 47), ('dit', 46), ('être', 46), ('c'est', 43),
('si', 39), ('gens', 39), ('faire', 36)]
--- Most common Words - Contradiction ---
[('a', 270), ('jamais', 151), ('n'a', 124), ('n'y', 88), ('tout', 85), ('plus', 68), ('tous', 58),
('aucun', 58), ('rien', 55), ('n'est', 52)]
--- Most common Words - Neutral ---
[('a', 266), ('plus', 123), ('qu'il', 65), ('parce', 62), ('si', 62), ('gens', 59), ('être', 56), ('tous', 47), ('beaucoup', 46), ('j'ai', 42)]
```

FIGURE 2 – Mots les plus fréquents par classe.

Préprocessing du dataset d'apprentissage.

En terme de nettoyage des données, nous n'en avons pas fait. En effet, les modèles sont entraînés à reconnaître le langage naturel donc nous n'avons pas eu besoin de supprimer les mots vides et la ponctuation.

2 Description de la méthode LoRA.

La technique LoRA pour Low-Rank Adaptation est une technique de fine-tuning. La technique de fine-tuning traditionnelle implique la mise à jour de tous les paramètres d'un modèle préentraîné afin de l'adapter à une tâche spécifique. Cependant, les modèles préentraînés peuvent compter des millions de paramètres. Donc cette technique implique un coût computationnel important. La technique LoRA propose une autre façon de faire qui consiste à insérer des adaptateurs dans les transformers. En effet, on les insère après le feed forward network et avant le layer normalization. Ce sont ces adaptateurs qui sont alors mis à jour.

Plus concrètement, cette méthode repose sur le fait que la matrice de paramètres d'un modèle préentraîné $W \in \mathbb{R}^{d \times k}$ peut être écrite comme le produit de deux matrices A et B tq $A \in \mathbb{R}^{d \times r}$ et $B \in \mathbb{R}^{r \times k}$ tq $r \ll d$ et $r \ll k$. A et B sont des matrices de faible rang donc plus facile à entraîner : ce sont les adaptateurs. À la fin du fine-tuning, on obtient : $W_{merged} = W + AB$. Au final, très peu de paramètres sont mis à jour. Cela réduit énormément le coût computationnel.

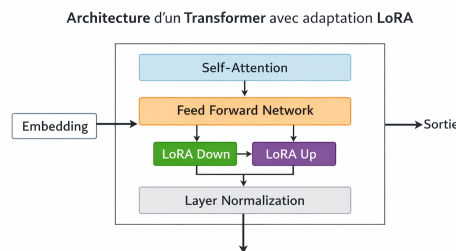


FIGURE 3 – Adapter Layer in LoRA.

3 Description des modèles préentraînés.

Modèle CamemBERT.

L'encodeur CamemBERT vient de la famille des encodeurs BERT. Comme vu au cours ce sont des MLM (Masked Language Model) qui comprennent le contexte d'une phrase en regardant les mots autour. Donc ils sont bidirectionnels d'où le B de BERT. De fait, BERT a été entraîné en se basant sur la concaténation de plusieurs langues. CamemBERT est donc une version BERT entraîné uniquement sur des données françaises. Cependant le CamemBERT ne vient pas directement de BERT mais de RoBERTa (MARTIN et al.) qui est une version beaucoup plus dynamique de BERT (Les mots sont masqués de façon dynamique.). De plus pour son apprentissage, le Next Sentence Prediction a été retiré.

Le CamemBERT se démarque des encodeurs multilinguistes car il n'a été entraîné que sur des données françaises. En outre, c'est un modèle qui s'adapte très bien à la classification des phrases paires. D'où son intérêt dans notre projet.

Modèle CamemBERTa.

L'encodeur CamemBERTa vient également de la famille des encodeurs BERT. Cependant, contrairement au CamemBERT, il ne vient pas du RoBERTa mais du DeBERTa v3 (ANTOUN, SAGOT et SEDDAH). Le DeBERTa v3 se distingue par l'intégration de deux techniques.

- L'Attention désagrégée qui consiste à traiter le mot et sa position séparément. C'est beaucoup plus précis pour comprendre la structure d'une phrase.
- Le Replaced Token Detection (RTD) : Cette notion vient du modèle Electra. Cela fonctionne avec deux réseaux de neurones. Le premier réseau génère les mots masqués. Le deuxième réseau détecte les erreurs (imposteurs) et les mentionne. Cela permet au modèle d'exploiter 100% du texte. Donc le modèle apprend plus vite.

Modèle LLaMA.

Le modèle LLaMA est un décodeur contrairement aux modèles CamemBERT. Il s'agit d'un modèle génératif. Il n'a donc pas le droit de voir les mots futurs mais juste les mots précédents pour générer le mot suivant. On parle d'un modèle de langue autorégressif. Il est généralement utilisé comme un modèle de discussion. Cependant, dans ce projet nous l'avons adapté à une tâche d'analyse de texte. Il s'agit d'un bon exercice pour déterminer les divergences entre les encodeurs et les décodeurs.

4 Description des modes d'apprentissage en contexte.

Mode 0-shot.

Ce mode d'apprentissage en contexte consiste à demander à un modèle préentraîné, à classer un texte en se basant juste sur sa compréhension sémantique. Et ceci sans avoir reçu aucun exemple dans son prompt.

Mode few-shot.

Ce mode est semblable au mode 0-shot. Sauf que cette fois-ci, on donne au modèle quelques exemples.

Mode « Chain-of-Thought ».

Comme le nom l'indique, on demande au modèle de décomposer son raisonnement avant la prédiction du mot suivant. Cela reconstitue un contexte qu'il va exploiter pour l'étape suivante. Cela évite que le modèle donne des réponses hâtives.

5 Implémentation

Dans cette section, nous allons énumérer les hyperparamètres des classes que nous avons utilisé pour les différents apprentissages.

Méthode LoRA.

Pour les trois modèles que nous avons fine-tuné avec la méthode LoRA, nous avons utilisé la même configuration. Cependant, selon le modèle il peut y avoir une différence par rapport aux couches où les matrices de rang faible sont injectées.

- $r = 8$: c'est le rang des matrices qui seront insérées dans les transformers. Plus ce rang est faible, moins le problème est complexe.
- $lora_dropout = 0.05$: spécifie le pourcentage de neurones qui sont désactivés.
- $task_type = "SEQ_CLS"$: cet hyperparamètre spécifie la tâche à accomplir.

Avec la méthode LoRA, le nombre de paramètres entraînables de chaque modèle est résumé dans le tableau ci-dessous.

Modèles	Nbre params entraînables	Nbre total params	%params entraînables
CamemBERT	887,811	112,490,502	0.7892
CamemBERTa	899,331	112,108,806	0.8022
Llama 3 1B	5,642,240	1,241,462,784	0.4545

TABLE 1 – Récapitulatif du nombre de paramètres entraînables

Le tableau ci-dessus illustre très bien la méthode LoRA. Les paramètres entraînés représentent moins de 1% du nombre total des paramètres des modèles préentraînés.

Arguments d'apprentissage du fine-tuning.

Les valeurs des arguments sont également les même pour les trois modèles.

- $learning_rate = 3e - 4$: le taux d'apprentissage.
- $weight_decay = 0.01$: la technique de régularisation.
- $optim = "adamw_torch"$: l'algorithme d'optimisation.

Apprentissage en contexte.

Pour l'apprentissage en contexte, nous utilisons un modèle "Llama 3 3B-Instruct". La mention Instruct indique que le modèle a été en plus entraîné pour répondre aux questions. Le code pour les trois modes d'apprentissage en contexte définis à la section précédente reste le même. La seule différence est au niveau de la génération du prompt. On dit au modèle ce qu'il doit faire selon le mode. De plus pour le mode « Chain-of-Thought », nous avons fixé le nombre maximal de token à générer à 150. Cela permet au modèle d'exprimer les étapes de son raisonnement.

6 Présentation des résultats

Résultats des tests avec la méthode LoRA.

Pour des contraintes computationnelles, nous avons fixé le nombre d'epochs à 4. Cependant, malgré un faible nombre d'epochs, les modèles ont bien performé.

Epochs	Epoch 1	Epoch 2	Epoch 3	Epoch 4
CamemBERT 2.0	0.410442	0.634538	0.691968	0.700000
CamemBERTa 2.0	0.635743	0.775904	0.779116	0.788353
Llama 3 1B	0.736948	0.784337	0.802008	0.795984

TABLE 2 – Tableau récapitulatif des Accuracy par epoch

Résultats des tests avec les méthodes d'apprentissage en contexte.

En plus de l'Accuracy, il es très intéressant d'analyser la f1-score par classe afin de voir la performance de ces modes dans le prédiction des différentes classes.

Modes	Accuracy	F1-score "Entailment"	F1-score "Contradiction"	F1-score "Neutral"
0-shot	0.4775	0.56	0.59	0.00
Few-shot	0.5570	0.62	0.74	0.02
Chain-of-thought	0.5394	0.59	0.70	0.27

TABLE 3 – Tableau récapitulatif des Accuracy et des F1-score

7 Analyse des résultats

Résultats des tests avec la méthode LoRA.

D'après le tableau (2), il est évident que les modèles CamemBERTa 2.0 et Llama 3 1B sont les plus performants. De plus dès la première epoch, ils affichent de très bons résultats. En ce qui concerne le CamemBERT 2.0 et le CamemBERTa 2.0, nous observons un très grand saut entre l'accuracy de l'epoch 1 et l'accuracy de l'epoch 2. Ce qui n'est pas le cas pour le Llama 3 1B. Nous pouvons l'interpréter comme une robustesse du Llama 3 1B par rapport aux modèles CamemBERT.

D'autre part, en analysant les f1-scores de chaque classe (confère notebook.ipynb), nous pouvons dégager le constat suivant. De façon générale, les trois classes ont été bien prédites. Cependant, au centième près la classe "neutral" a un f1-score moindre que les autres classes. Cela signifie que les modèles ont un peu de mal à prédire cette classe.

Résultats des tests avec les méthodes d'apprentissage en contexte.

En analysant le tableau (3), le mode Few-shot s'est avéré le plus performant. Ensuite nous avons le mode Chain-of-thought qui suit. Il est cependant assez difficile de départager ces deux modèles car pour le mode Chain-of-thought, nous avons dû spécifier le nombre maximal de token à générer. Cet hyperparamètre peut en effet, limiter les performances de ce dernier. De façon générale, les performances de l'apprentissage en contexte restent moindres par rapport au fine-tuning.

Par ailleurs en analysant les f1-scores, il est évident qu'en mode question-réponse les modèles ont dû mal à prédire la classe "neutral". Le seul mode ayant pu légèrement capté cela est le mode Chain-of-thought.

8 Conclusion

Somme toute, dans ce projet, nous avons testé les modèles CamemBERT 2.0, CamemBERTa 2.0 et LLaMA 3. Nous avons commencé par une exploration des données, qui a montré que les classes étaient bien équilibrées et qu'aucune anomalie n'était présente.

Ensuite, nous avons réalisé une veille documentaire sur les différents modèles, ce qui nous a permis de mettre en évidence les différences entre CamemBERT et CamemBERTa. CamemBERT est basé sur RoBERTa, qui est une variante de BERT sans "Next Sentence Prediction" et utilisant un Masked Language Modeling dynamique. En revanche, CamemBERTa repose sur DeBERTa v3, qui se distingue du BERT par une Attention désagrégée et le Replaced Token Detection (RTD).

Nous avons ensuite fine-tuné nos modèles en utilisant la méthode LoRA, qui repose sur l'utilisation de matrices de rang faible pour réduire le nombre de paramètres à entraîner lors du fine-tuning. Grâce à cette approche, les modèles CamemBERTa 2.0 et LLaMA 3 1B se sont montrés particulièrement performants.

Par la suite, nous avons exploré trois modes d'apprentissage en contexte qui sont le 0-shot, le few-shot et le Chain-of-thought, en les appliquant au modèle LLaMA 3 3B-Instruct. Nous avons constaté que leurs performances étaient généralement inférieures à celles obtenues avec le fine-tuning, notamment pour la prédiction de la classe "neutral". Le mode Chain-of-thought s'est toutefois distingué sur cette tâche, car le fait d'étaler le raisonnement a permis d'améliorer nettement la prédiction de la classe "neutral".

Références

- [1] Wissam ANTOUN, Benoît SAGOT et Djamé SEDDAH. "Data-Efficient French Language Modeling with CAMEMBERTA". In : *Inria, Paris*, (2021).
- [2] Louis MARTIN et al. "CamemBERT : a Tasty French Language Model". In : *hal-02889805*, (2020).